

Ecole Nationale des Sciences Appliquées - Al Hoceima
Travaux pratiques N°2 : L'algorithme Minimum Edit Distance et Autocorrection

Module : Traitement Automatique de la Langue Naturelle (NLP)

Préparé par : Pr. Tarik BOUDAA

Exercice 1

- 1- Implémenter l'algorithme de distance d'édition minimale vu dans le cours.
- 2- Enrichissez votre fonction de distance d'édition pour également enregistrer le chemin de retour en arrière (*backtrace*) comme décrit dans le cours. Utilisez ce chemin pour trouver un alignement entre les chaînes de caractères.
- 3- Coûts d'adjacence. Les coûts dans ce calcul sont finalement des décisions arbitraires. Essayez de modifier les fonctions de coût pour calculer une version plus intelligente de la distance d'édition qui pourrait être applicable à la correction orthographique. Il existe plusieurs façons de le faire. On pourrait exploiter la remarque suivante par exemple : le coût de substitution pour frapper une touche adjacente est plus bas que le coût de frapper une touche non adjacente. En supposant un clavier QWERTY vous pouvez utiliser les données indiquées dans le fichier *qwerty_graph.txt*
- 4- Implémenter votre première version d'un correcteur d'orthographe en se basant sur l'algorithme simple suivant :
Soit w un mot contenant une erreur d'orthographe (un mot absent du dictionnaire), calculer la distance entre w et tous les mots du dictionnaire contenant les mots valides puis proposer les k corrections possibles (les k plus proches mots selon la distance d'édition minimale).
Vous pouvez utiliser le dictionnaire français donnée dans le fichier *dictionnaire_fr.txt*.
- 5- Optimiser l'algorithme précédent en prenant en compte les remarques suivantes :
 - Les gens ont généralement la première lettre du mot correcte, donc nous pouvons limiter notre recherche aux mots commençant par la même lettre.
 - Nous pouvons restreindre notre recherche aux mots de même longueur ou de longueur similaire.
 - Nous pouvons restreindre notre recherche aux mots qui ont le même son, en utilisant un code phonétique pour regrouper les mots (comme Soundex).
 - En utilisant d'autres techniques de votre choix

Exercice 2

On considère le problème de la détermination de la correction la plus probable d'un mot introuvable en dictionnaire. Ainsi, le problème est de trouver la correction c , parmi toutes les corrections candidates possibles, qui maximise la probabilité que c soit la correction voulue, étant donné le mot original w : $\operatorname{argmax}_{c \in \text{candidates}} P(c|w)$. Par le théorème de Bayes, cela équivaut à :

$$\operatorname{argmax}_{c \in \text{candidates}} P(c) P(w|c) / P(w)$$

Étant donné que $P(w)$ est la même pour chaque candidat possible c , nous pouvons l'éliminer, ce qui donne : $\operatorname{argmax}_{c \in \text{candidates}} P(c) P(w|c)$

Les éléments de cette équations sont :

- Modèle de langage $P(c)$: la probabilité que c apparaisse en tant que mot dans un texte en anglais.
 - Modèle d'erreur $P(w|c)$: la probabilité que w soit tapé dans un texte lorsque l'auteur voulait écrire c . Par exemple, $P(teh|the)$ est relativement élevée, mais $P(theexyz|the)$ serait très faible.
- 1- Implémenter la fonction *process_data(corpus_file)* qui permet de lire le corpus donné sous forme d'un fichier texte, convertir le texte en minuscule et segmenter le texte et retourne la liste des mots.
 - 2- Implémenter la fonction *get_vocabulary(corpus_file)* qui retourne le vocabulaire construit à partir d'un corpus passé en argument de la fonction.
 - 3- Nous pouvons estimer la probabilité d'un mot, en comptant le nombre de fois où ce mot apparaît dans un corpus de texte de grande taille et en divisant sur la taille de corpus. Ecrire une fonction qui construit le modèle de langue en calculant la probabilité de chaque mot en se basant sur le fichier *big.txt* fournit avec ce TP et elle stocke le résultat dans une structure de données adéquate. N'oubliez pas d'effectuer le preprocessing avec la fonction *process_data*.
 - 4- Ecrire les fonctions suivantes :
 - *edits1(s)* : qui retourne l'ensemble de toutes les chaînes (qu'il s'agisse de mots ou non) qui peuvent être obtenues avec une seule modification (insertion, substitution ou suppression) à effectuer sur la chaîne s .
 - *edits2(s)* : qui retourne l'ensemble de toutes les chaînes (qu'il s'agisse de mots ou non) qui peuvent être obtenues avec 2 modifications (insertion, substitution ou suppression) à effectuer sur la chaîne s .
 - *knownWord(words)* qui filtre les mots de la liste *words* qui ne sont pas dans le vocabulaire (elle garde donc que les mots valides). On peut utiliser la fonction *get_vocabulary(corpus_file)*
 - 5- On suppose que nous n'avons pas des données pour construire le modèle d'erreur, ainsi nous allons adopter les hypothèses suivantes: tous les mots connus avec une distance d'édition de 1 sont infiniment plus probables que les mots connus avec une distance d'édition de 2, et infiniment moins probables qu'un mot connu avec une distance d'édition de 0. Ainsi, pour sélectionner les candidats les plus probables nous considérons leurs probabilités selon le modèle de langue préalablement construit et leurs priorités en fonction de leur distance d'édition du mot original. Avec cette simplification, nous n'avons pas besoin de multiplier par un facteur $P(w|c)$, car chaque candidat à la priorité choisie aura la même probabilité. Ecrire la fonctions *candidates(word)* qui retourne la première liste non vide de candidats par ordre de priorité :
 - Le mot original, s'il est connu ; sinon*
 - La liste des mots connus à une distance d'édition de un, s'il y en a ; sinon*
 - La liste des mots connus à une distance d'édition de deux, s'il y en a ; sinon*
 - Le mot original, même s'il n'est pas connu.*
 - 6- En utilisant les fonctions précédente, écrire la fonctions *correction(word, k)* qui retourne les k plus probables corrections du mot *word*.

Références :

Speech and Language Processing : <https://web.stanford.edu/~jura/sky/slp3/2.pdf>
Peter norvig : <https://norvig.com/spell-correct.html>